

QUESTION 01: DATABASE SCHEMA & PERFORMANCE OPTIMIZATION

1. Database Schema Design (Scale: 1M Products & 500,000 Users)

At a scale of **1,000,000 products** and **500,000 users**, an enterprise-grade Relational Database Management System (RDBMS) such as **PostgreSQL** or **MySQL** is required. This ensures strict ACID compliance, transactional integrity for order processing, and robust indexing capabilities.

Table Name	Key Attributes / Fields	Primary Key	Constraints
users	user_id, email, password_hash, full_name, role, created_at	user_id	NOT NULL, UNIQUE, INDEXED
categories	category_id, name, slug, parent_category_id	category_id	NOT NULL, INDEXED
products	product_id, title, slug, description, price, stock_quantity, category_id, created_at	product_id	NOT NULL, INDEXED
orders	order_id, user_id, status, total_amount, created_at	order_id	NOT NULL, INDEXED
order_items	order_item_id, order_id, product_id, quantity, unit_price	order_item_id	NOT NULL, INDEXED

2. How Indexing Improves Application Performance

Indexing generates auxiliary self-balancing data structures (typically **B-Trees**) that allow the database engine to locate records in logarithmic time $O(\log N)$ instead of performing a linear $O(N)$ Full Table Scan across millions of rows.

- Primary & Unique Indexes:** Automatically indexed fields like `product_id`, `email`, and `slug` enable instant point lookups.
- Foreign Key Indexing:** Indexing foreign keys (e.g., `category_id` in `products`, `user_id` in `orders`) drastically optimizes high-frequency relational `JOIN` operations.

- **Composite Indexes:** Multi-column indexes such as `(category_id, price)` allow the database to handle catalog filtering and price sorting simultaneously without standard in-memory re-sorting.
- **Full-Text Indexing:** Applied to product titles and descriptions to accelerate user search queries without degrading throughput.

3. How Normalization Improves Performance & Data Integrity

Structuring data up to **Third Normal Form (3NF)** eliminates operational anomalies and redundant data storage:

- **Elimination of Data Redundancy:** Storing category names inside every single product entry would consume excessive storage across 1 million products. Normalizing categories into a dedicated table ensures attributes are stored exactly once.
- **Optimized Write Operations:** Modifying a category name requires updating a single record in `categories` rather than locking and updating hundreds of thousands of product rows.
- **Data Integrity & Anomalies:** Prevents insertion, update, and deletion anomalies, maintaining strict consistency across customer orders and catalog records.

Production Architecture Note: For high-traffic read operations, relational normalized databases are complemented with read-replicas, in-memory caching (Redis) for hot product data, and specialized search engine indices (Elasticsearch/Algolia).

QUESTION 02: WEB APPLICATION SECURITY VULNERABILITIES & MITIGATION

1. Cross-Site Scripting (XSS)

Threat: An attacker injects malicious JavaScript code into user-submitted fields (such as product reviews or customer profile details). When rendered in another user's browser, the script executes, enabling session token theft or page defacement.

Prevention & Secure Practices:

- **Automatic JSX Escaping:** React automatically escapes strings embedded in JSX variables prior to rendering, preventing standard HTML script injection.
- **Avoid Dangerous Hooks:** Refrain from using `dangerouslySetInnerHTML`. If rich HTML rendering is strictly necessary, sanitize inputs using robust libraries like `DOMPurify`.
- **HttpOnly Cookie Storage:** Store authentication JWTs in `HttpOnly` and `Secure` cookies so client-side JavaScript cannot access them via `document.cookie`.
- **Content Security Policy (CSP):** Enforce HTTP headers that specify whitelisted script execution domains.

2. Cross-Site Request Forgery (CSRF)

Threat: An authenticated user visits a malicious website, which silently triggers unauthorized requests (e.g., placing an order or updating profile details) to the e-commerce backend using the user's active session cookies.

Prevention & Secure Practices:

- **SameSite Cookie Attribute:** Configure all session/auth cookies with `SameSite=Strict` or `SameSite=Lax` to prevent browsers from attaching cookies to cross-site requests.
- **Anti-CSRF Tokens:** Implement unique, cryptographically secure anti-CSRF tokens for state-changing requests (`POST` , `PUT` , `DELETE`).
- **Strict CORS Configuration:** Whitelist only authorized client origins on the backend CORS middleware.

3. SQL Injection (SQLi)

Threat: Unsanitized input supplied via search bars or login forms alters the backend SQL syntax, allowing attackers to execute arbitrary queries, read sensitive data, or delete database tables.

Prevention & Secure Practices:

- **Parameterized Queries / ORMs:** Always use parameterized statements or ORMs (e.g., Prisma, Sequelize, Knex.js) that treat inputs as literal values rather than executable code.
- **Avoid String Concatenation:** Never construct raw SQL strings via direct string concatenation with user input variables.

4. Broken Authentication & Session Vulnerabilities

Threat: Weak credential storage or unthrottled authentication routes allow attackers to perform brute-force password guessing or credential stuffing attacks.

Prevention & Secure Practices:

- **Strong Cryptographic Hashing:** Hash all user passwords using adaptive algorithms like `Bcrypt` (with appropriate work factors) or `Argon2` before database persistence.
- **Rate Limiting & Throttling:** Apply middleware rate-limiters (e.g., `express-rate-limit`) on authentication endpoints (`/api/login` , `/api/reset-password`) to block automated brute-force attempts.